

JobMatch: An Agentic Multi-Role Platform for Explainable Job–Candidate Matching

Harsh Kashyap^{1*} and Taranumpreet Kaur Wasu¹

^{1*}Thapar Institute of Engineering and Technology, Patiala, India.

Abstract

Job seekers and recruiters frequently receive ranked lists without clear attribution of profile ownership, scoring rationale, or control over consequential actions. We present JobMatch, a multi-agent recruitment platform in which Candidate and Employer agents maintain role-specific profiles and embeddings, while a read-only Matchmaking Agent scores and explains candidate–job pairs. Agents coordinate through an event bus; when a résumé or job description changes, the owning agent updates vector representations and notifies the Matchmaking layer to refresh stale rankings. Separate candidate, employer, and admin portals keep save, apply, and shortlist decisions with human users. For deployment, the Employer Agent can optionally refresh job postings from an external provider API; all offline metrics in this paper use the frozen demo corpus.

The Matchmaking agent combines semantic similarity, skill overlap, and experience, compensation, and remote compatibility into a fixed composite score with structured explanations—assistive ranking, not autonomous hiring. We evaluate offline on 30 resumes, 15 job postings, and 47 graded relevance pairs using macro-averaged P@5, R@5, and nDCG@5 at $K=5$. Compared methods include TF-IDF and BM25 baselines, dense retrieval, reciprocal rank fusion, learned fusion, and cross-encoder reranking. Multimodal soft-embed and learned fusion reach nDCG@5 of 0.969 and 0.968 in committed artifacts; composite ablation reaches 0.942 under a documented weight vector. Cross-encoder reranking reduces nDCG@5 by 0.108 at ~ 141.7 ms/query and remains disabled in the default portal. Hard-negative mining yields 150 pairs with zero label conflicts; a synthetic fairness audit flags 6/10 fabricated pairs—an engineering check, not demographic validation.

Benchmark drivers and regression tests reproduce reported metrics. We document multi-agent architecture, implementation, quality metrics, and results as an integrated research prototype, not a validated production hiring system.

Keywords: multi-agent systems, recruitment platforms, explainable matching, job–candidate ranking, hybrid retrieval

1 Introduction

Online hiring markets route high application volume through platforms that rank candidates and jobs at scale, yet both sides often receive limited feedback on why a match appeared, changed, or disappeared. Candidates submit the same résumé to many openings and cannot distinguish rejection, filtering, and queue effects. When outcomes arrive—an interview invitation, a rejection, or a role removed from a results page—the rationale seldom returns to the applicant in actionable form.

Recruitment platforms centralize workflow, but they rarely make ranking decisions auditable. Candidates see ordered lists without evidence tying each slot to profile fields or job requirements. Recruiters and hiring managers see shortlists update after job-text edits without a visible link between requirement changes and rank movement. Neither side can reliably answer: *why was this person shown this role, and what would need to change for the ranking to differ?*

These gaps impose measurable cost. Misaligned interviews consume calendar time on both sides; qualified candidates who never reach a shortlist extend vacancies or divert talent elsewhere. Organizations pay in recruiter hours and manager attention; job seekers pay in repeated preparation for roles that were poor fits from the outset.

Assistive artificial intelligence can address such opacity without automating hire-or-reject decisions. Sal Khan [1] argues that AI should guide people through complex choices—clarifying trade-offs, supporting preparation, and preserving human judgment. In admissions and hiring alike, static snapshots often replace evolving context; software that maintains structured, versioned profiles can instead support informed decisions over time.

We instantiate that principle with a dedicated agent on each market side. A **candidate-side agent**—also referred to as the client-side agent in our implementation—accepts résumés and profile updates, extracts a structured record of skills and experience, and keeps that record current as the user revises materials. An **employer-side agent** performs the parallel work for hiring: it accepts job descriptions and requirements, organizes them into a structured job record, and updates that record when a role changes. Each agent owns its side of the market; neither manages the other’s data.

Separate ownership only helps if the two sides can still meet on common ground. When candidate records and job records never connect, both portals collapse into blind browsing. When one large program controls every edit and every score, users cannot see who changed what or why a recommendation moved. JobMatch therefore treats **collaboration among agents** as the core design problem. Candidate and employer agents maintain their own state; a neutral **Matchmaking Agent** reads both, compares them, and returns ranked recommendations with structured explanations. When either side updates a résumé or a job description, the owning agent refreshes its record and notifies the Matchmaking layer so stale rankings can be invalidated.

These components form **JobMatch**, a multi-agent job-matching platform implemented as a research prototype with separate web portals for candidates, employers, and administrators. The platform parses résumés and job descriptions into structured states, builds comparable representations of people and roles, and routes matching

through the Matchmaking Agent to produce ordered recommendations. For deployment scenarios, the Employer agent can optionally refresh its job corpus from an external provider API; offline evaluation in this paper uses a fixed demo corpus (§4). Users remain in control throughout: they review extracted fields, read why a pairing ranked as it did, and decide whether to save, apply, or shortlist. The system advises; it does not hire or reject on anyone’s behalf. We evaluate the matching core offline on a labeled demo corpus—30 résumés, 15 jobs, and 47 graded relevance pairs—using standard ranking metrics and reproducible benchmark comparisons (§5–§6). Hybrid rankers reach nDCG@5 up to 0.969 on this corpus; the portal defaults to a fixed composite score whose components map to user-visible explanations.

This paper makes the following contributions:

- A **Candidate/Client Agent** that ingests résumés, maintains versioned candidate state, and updates candidate embeddings on confirmed saves.
- An **Employer Agent** that ingests job descriptions, maintains versioned job state, and optionally refreshes postings from an external provider API.
- A **Matchmaking Agent** that scores candidate–job pairs read-only and returns ranked lists with structured explanations.
- An **event-driven communication model** in which profile updates invalidate stale rankings without cross-agent writes to foreign stores.
- A **role-separated portal layer** demonstrating end-to-end candidate, employer, and administrator workflows.
- An **offline evaluation** on a fixed labeled corpus with benchmark comparisons, ablations, and a continuous-integration regression gate.

The remainder of the paper is organized as follows. Section 2 reviews recruitment systems, AI agents in hiring, and related matching approaches. Section 3 presents the multi-agent architecture (Candidate, Employer, and Matchmaking agents; communication; overall workflow). Section 4 describes implementation details (libraries, APIs, data structures, and parameters). Section 5 defines quality metrics, similarity scores, and evaluation criteria. Section 6 reports benchmark results and discussion. Section 7 concludes and outlines future scope.

2 Literature Review

Recruitment technology, agentic software, and information retrieval have each advanced independently, yet deployed hiring tools rarely combine all three in a traceable architecture. We organize prior work into five literature themes—recruitment systems, multi-agent coordination, semantic representation, ranking and fusion—then state the gaps those literatures leave open and how JobMatch is positioned relative to them. Numeric results from our evaluation appear in §6; metric definitions in §5.

2.1 Recruitment and job-matching systems

Enterprise hiring still depends heavily on manual screening: recruiters read resumes, compare them to written requirements, and decide who advances. At scale this workflow is slow, inconsistent across reviewers, and weakly auditable when a candidate asks why they were excluded *TODO*: Citation needed: empirical studies of manual resume screening throughput and reviewer inconsistency in recruitment. Most production stacks therefore automate the first cut through keyword filters, structured application forms, and applicant tracking systems (ATS).

Keyword-based filtering treats a resume as a bag of terms matched against job requirements. Lexical retrieval methods such as TF-IDF weighting and BM25 remain the workhorse for this pattern because they are fast, interpretable at the token level, and easy to deploy over large applicant pools [2, 3]. Their limitation is semantic brittleness: synonyms, paraphrases, and transferable skills that are not spelled out literally can be missed, while keyword stuffing can inflate apparent fit. Structured skill fields and Boolean search rules reduce noise but still encode fit as a static rule set rather than a graded comparison between two evolving profiles.

ATS platforms extend ingestion with workflow state—application status, interview stages, compliance logging—but they often treat matching as a side feature bolted onto document storage *TODO*: Citation needed: survey or industry analysis of applicant tracking system capabilities and matching modules. Candidates frequently experience opaque ranking: a profile passes a filter without knowing which requirement mattered, or disappears after a re-rank with no explanation. Recruiters face the symmetric problem when shortlists shift without a visible link between job-text edits and ranking changes. These workflow limitations motivate systems that separate *profile ownership* from *scoring*, and that return reasons a human can inspect before any hire/no-hire action is taken.

2.2 AI agents and multi-agent systems

Agent-based software assigns bounded roles—perception, memory, planning, action—to modules that coordinate through messages or shared events rather than through one monolithic controller [4]. Three agent patterns are relevant to hiring.

Personal agents.. Personal or client-side agents maintain a durable model of a user’s goals and history, updating that model as new documents arrive *TODO*: Citation needed: personal software agents or digital assistants for career and document management. In hiring, the natural analogue is a representative that ingests resumes, tracks skill and experience revisions, and speaks for the candidate without exposing raw edits to the employer’s datastore.

Task-specific agents.. Task-specific agents encapsulate one domain workflow—parsing, validation, or recommendation—with explicit inputs and outputs. Employer-side job-ingestion agents fit this pattern: they normalize postings, extract requirements, and version job records when descriptions change. Prior agentic hiring proposals often emphasize autonomous interview bots or negotiation agents *TODO*: Citation needed: autonomous interview or negotiation agents in recruitment. JobMatch instead uses task-specific agents for *profile maintenance*, not for autonomous selection.

Collaborative agents and human-in-the-loop support.. Collaborative multi-agent systems decompose a problem so that each agent owns part of the state and a coordinator integrates read-only views for decision support [4]. Human-in-the-loop designs keep consequential actions—apply, reject, shortlist—with people while software proposes ranked options and explanations *TODO*: Citation needed: human-in-the-loop decision support for algorithmic hiring recommendations. This differs from fully automated ranking pipelines that update applicant status without an explicit user action on each match.

JobMatch adopts the collaborative pattern: Candidate and Employer agents own their respective profiles and vector stores; a read-only Matchmaking agent scores cross-side pairs and emits structured explanations (§3). An in-process event bus synchronizes profile updates and match-cache invalidation without cross-agent mutation of stored records (§4).

2.3 Semantic search and representation learning

Job–candidate matching is, at core, a cross-document retrieval problem: given a resume (query), rank jobs (documents), or the reverse for employer-initiated search. Semantic search replaces exact term overlap with learned representations that map text into a shared vector space [2].

Embeddings.. Dense encoders map resumes and job descriptions to fixed-dimensional vectors so that related texts lie nearby even when they share few tokens. Sentence-level models such as Sentence-BERT train siamese encoders for semantic similarity and have become a default building block for short-text matching [5]. In JobMatch, snapshot embeddings are computed from canonical resume and job-description text (§4.5).

Dense retrieval and semantic similarity.. Dense retrieval ranks candidates or jobs by vector similarity—typically cosine—over the full corpus or over an approximate nearest-neighbor shortlist *TODO*: Citation needed: dense passage or document retrieval over learned embeddings. Compared with lexical baselines, dense retrieval captures paraphrase and topical relatedness but can underweight explicit skill tokens that recruiters treat as hard requirements. Hybrid systems therefore retain both signals rather than replacing keywords entirely [2, 5].

Much published job-matching research evaluates a single embedding model or similarity function on a static labeled set *TODO*: Citation needed: job–resume matching studies using neural or embedding-based retrieval. Fewer prototypes expose the same representation path to live profile edits on both market sides while reporting offline retrieval metrics on identical scoring code.

2.4 Ranking and recommendation

Once candidates and jobs are represented, the system must *rank* pairs and *evaluate* ranking quality—topics well developed in information retrieval and recommender-systems research.

Ranking metrics.. Standard offline metrics include precision and recall at cutoff K , mean reciprocal rank (MRR), normalized discounted cumulative gain (nDCG@ K), and

mean average precision (MAP) [2]. Precision@K measures shortlist purity; recall@K measures coverage of labeled relevant items; nDCG@K rewards placing higher-grade relevance labels earlier; MRR emphasizes the rank of the first relevant result. Graded nDCG is appropriate when relevance is partial (e.g., a job is a plausible but not ideal fit). We adopt these metrics at $K=5$ on a labeled demo corpus (§5).

Hybrid retrieval.. Hybrid rankers combine lexical scores, dense semantic similarity, and structured features such as skill overlap or experience gaps [2, 3, 5]. A common pattern is a weighted sum of semantic and skill channels (multimodal blending) or a post-hoc constraint filter on top of a semantic shortlist. JobMatch extends this with a multi-component composite score—semantic, skills, title overlap, experience, compensation, and remote preference—so that each channel remains visible in portal explanations (§5).

Fusion methods.. When multiple rankers produce different orderings, fusion methods merge lists without retraining the underlying encoders. Reciprocal rank fusion (RRF) combines ranked outputs using reciprocal rank weights and remains stable when individual retrievers make complementary errors [6]. Learned fusion and hierarchical blending fit scalar weights or classifiers over component scores *TODO*: Citation needed: learned rank fusion or stacking for multi-signal retrieval. JobMatch implements fixed-weight composite scoring for portal defaults, plus optional RRF, learned logistic fusion, Platt calibration [7], and cross-encoder reranking for offline comparison (§4, §6).

2.5 Research gaps

Across the recruitment, agent, and retrieval literatures, several gaps recur in job-matching demos and prototypes.

Algorithm-only focus.. Many systems optimize a matching formula—embedding similarity, skill Jaccard, or a learned re-ranker—while treating ingestion, profile versioning, and user-facing workflow as out of scope *TODO*: Citation needed: job-matching prototypes that report retrieval metrics without deployable profile or portal layers. Evaluation then proceeds on frozen corpora disconnected from the code path users would actually trigger when they edit a resume or repost a job.

Single-sided or monolithic control.. Candidate-side recommender demos are more common than symmetric platforms where *both* sides maintain agents with separate ownership *TODO*: Citation needed: two-sided or employer-candidate job-matching platforms with separate user roles. Monolithic services that store and score all records in one module obscure accountability: users cannot tell which component changed a rank or which side’s edit caused invalidation.

Limited explainability and evaluation depth.. Ranking scores alone do not satisfy recruiter or candidate trust; explainable ML methods attribute predictions to input features [8]. In hiring, explanations should reference skills, experience gaps, and constraint channels that align with the score breakdown. Yet many job-matching papers report nDCG or accuracy without explanation-quality audits, calibration checks [7], or fairness sanity tests on controlled profile variants *TODO*: Citation needed: explainability or fairness evaluation in job-matching or hiring recommender systems. Demo corpora are often small and manually labeled, which is appropriate for

reproducible research but rarely accompanied by regression gates tied to production scoring code.

2.6 Positioning of JobMatch

JobMatch addresses these gaps as an integrated research prototype rather than as a standalone retrieval formula.

Multi-agent framing.. The system decomposes hiring into three bounded agents—Candidate, Employer, and Matchmaking—coordinated by typed events (§3–§4). Profile mutation stays with the owning agent; matching reads versioned snapshots only. This design makes ownership, cache invalidation, and audit trails first-class, not emergent properties of a single service.

Candidate-side and employer-side portals.. Separate React portals enforce role separation: candidates manage resumes, saved jobs, and applications; employers manage postings, reverse candidate search, and applicant lists; administrators inspect agent status, run evaluation hooks, and review fairness summaries (§4.2). Consequential actions remain explicit user clicks—save, apply, dismiss—consistent with human-in-the-loop decision support.

Matchmaking agent and explainable composite ranking.. The Matchmaking agent combines semantic, skill, and constraint channels into a fixed composite score with structured `MatchExplanation` payloads (§5). Rule-based explainers are evaluated offline for faithfulness and specificity (§5.5); optional calibration and feedback boosts exist but are disabled in default portal settings.

Empirical evaluation on shared code paths.. Offline benchmarks call the same scoring core as the live gateway: lexical and dense baselines, multimodal and composite strategies, RRF and learned fusion, ablations, hard-negative sanity checks, synthetic fairness counterfactuals, and a continuous-integration regression gate on `nDCG@5` (§5–§6). We do not claim state-of-the-art hiring accuracy on a large production corpus; the contribution is traceable multi-agent architecture, assistive explainable ranking, and reproducible measurement on a controlled labeled set tied to the deployed prototype.

3 Architecture of the Multi-Agent System

JobMatch organizes hiring support around three collaborating agents and a shared representation layer. Figure 1 shows the high-level layout; Figures 2–4 expand each agent’s internal structure; Figure 5 summarizes the full block architecture; Figures 6 and 7 trace end-user workflows. Three design principles guide the layout: **separation of ownership** (each market side controls its own records), **read-only matchmaking** (scoring never edits profiles or acts on behalf of users), and **human-in-the-loop decisions** (save, apply, contact, and shortlist remain explicit user actions). Concrete libraries, APIs, and parameters appear in §4; score formulas and evaluation metrics in §5.

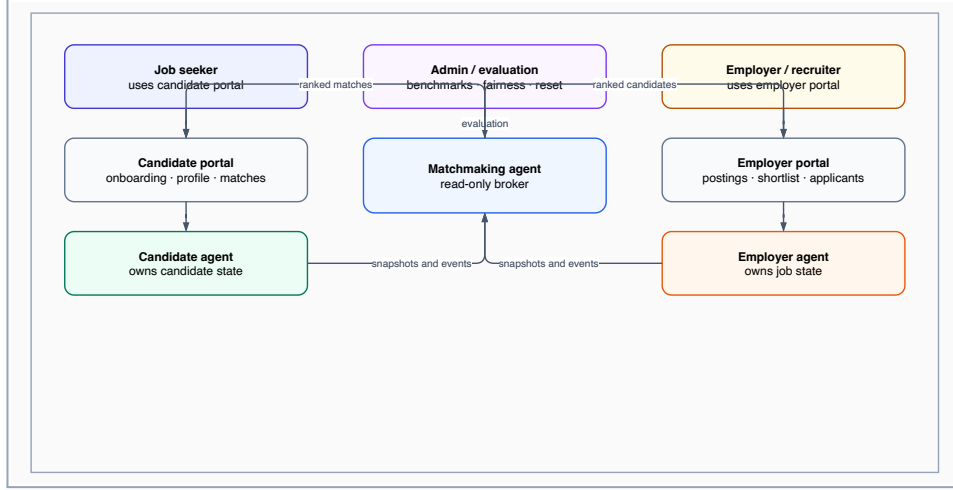


Fig. 1: High-level view of JobMatch: two market-side agents own candidate and job state; a read-only Matchmaking agent scores pairings and returns explained rankings to role-specific portals. Snapshots and invalidation events connect owning agents to the matcher; an admin console supports evaluation without crossing ingestion boundaries.

3.1 Candidate/Client Agent

The **candidate-side agent** (also referred to as the client-side agent in our implementation) represents the job seeker. Its responsibility is to turn resumes and profile edits into a maintained *candidate state* that the rest of the system can trust.

When a user uploads or revises a resume, this agent assists with parsing, normalizes skills into a shared vocabulary, and registers a versioned snapshot of the person’s skills, experience, compensation expectations, and remote preferences. It also maintains the editable portal profile the user sees in forms. The agent publishes a profile-updated signal whenever a new snapshot is ready so that stale match lists can be refreshed.

The candidate agent does *not* own job postings, write into employer records, or decide which applications are sent. Those boundaries keep accountability clear: the seeker’s representative manages seeker data only.

3.2 Employer Agent

The **employer-side agent** represents the hiring organization. It accepts job descriptions and requirement updates, structures them into a maintained *job state*, and keeps that state aligned with what recruiters edit in the employer portal.

For each posting, the agent registers a snapshot with required and preferred skills, experience bounds, compensation band, remote policy, and descriptive text suitable for comparison against candidates. Like its counterpart on the candidate side, it emits a job-updated signal when a snapshot changes so match rankings can be invalidated without cross-agent writes.

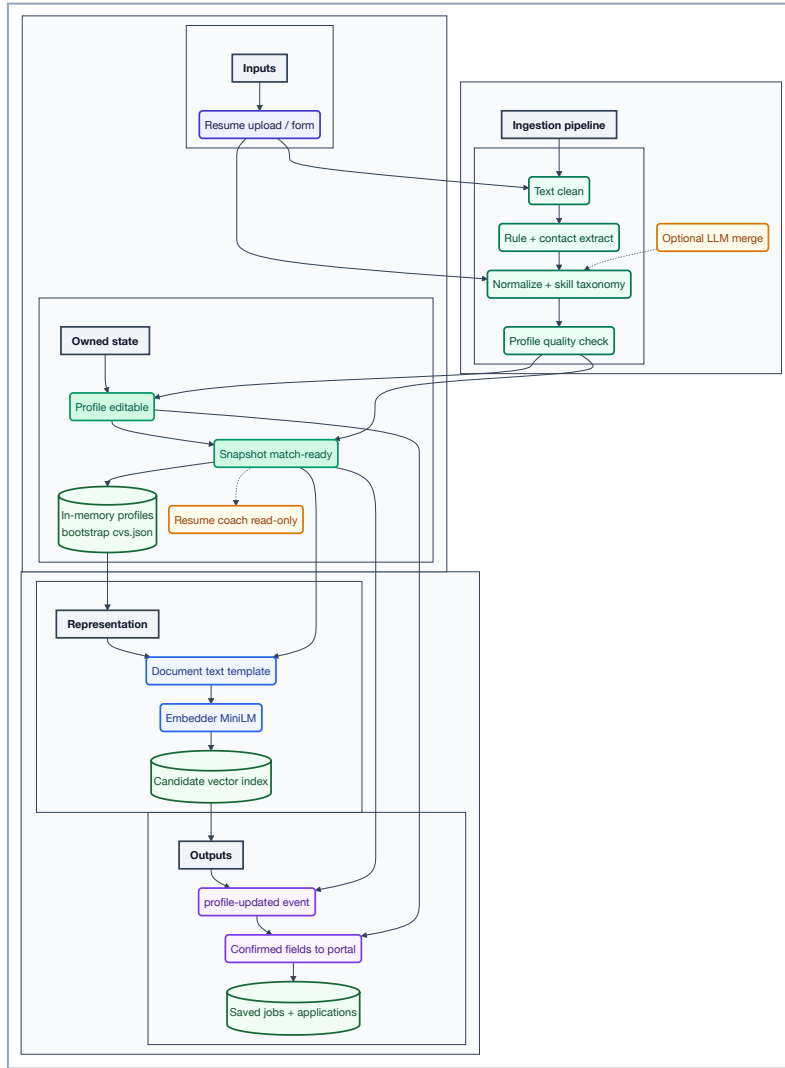


Fig. 2: Internal structure of the Candidate/Client agent: document ingestion (clean, rule extract, optional LLM merge), skill normalization, profile quality check, editable profile versus match-ready snapshot, embedding, and side outputs (events, activity store, resume coach).

The employer agent does *not* manage candidate profiles or override match scores. Shortlisting and applicant review remain human decisions supported by ranked suggestions.

Optional external job feed.. When enabled at deployment time, the Employer agent can also ingest paginated job listings from an external provider API. Each raw

record is normalized to the same canonical job schema used for portal postings, deduplicated by stable identifier, and written to a local snapshot file for restart recovery. A sync operation replaces the in-memory employer corpus and re-embeds job snapshots; the frozen `jobs.json` demo seed remains the default for offline evaluation reported in §6. The Matchmaking agent consumes the refreshed snapshots read-only; it does not call the external provider directly.

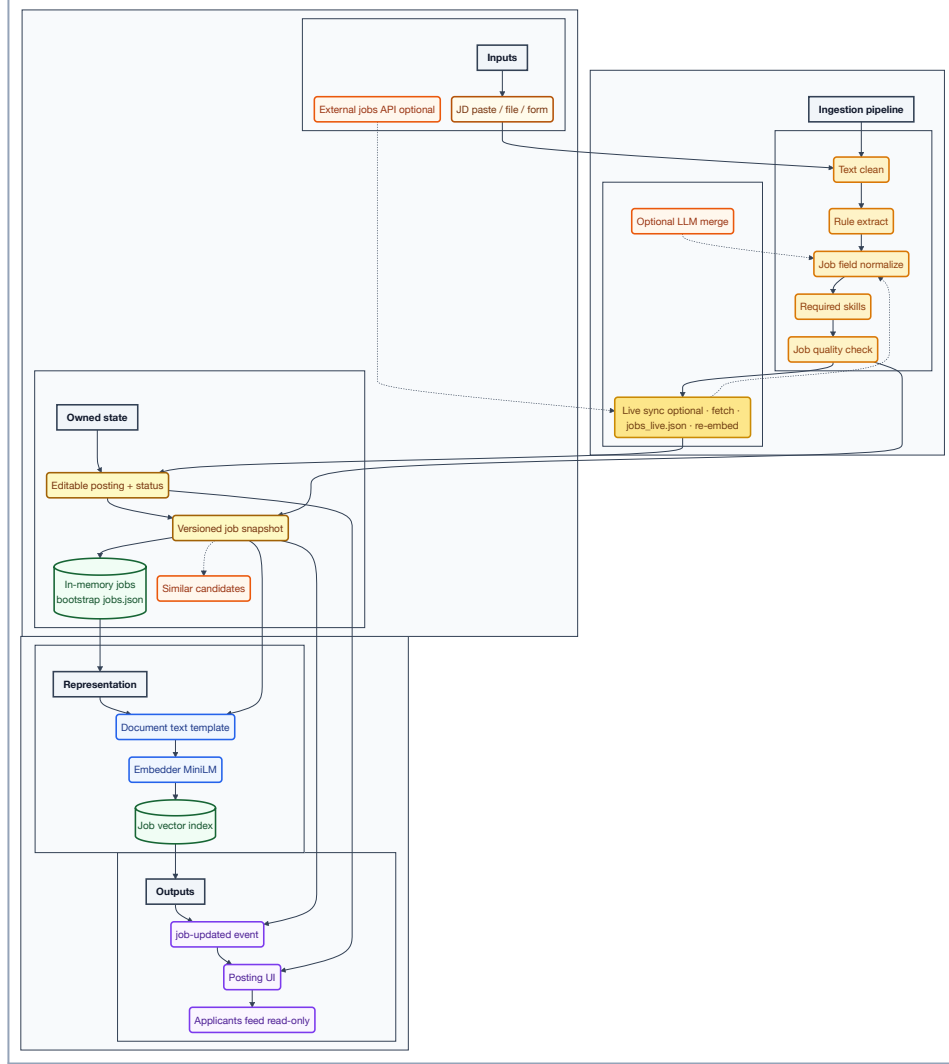


Fig. 3: Internal structure of the Employer agent: JD ingestion (clean, rules, optional LLM), job quality check, posting status lifecycle, optional external feed sync, job snapshots, and side outputs (events, applicants feed, similar candidates).

3.3 Matchmaking Agent

The **Matchmaking Agent** is a neutral broker between the two market sides. Given a candidate snapshot and a set of job snapshots—or the reverse direction for employer-initiated search—it scores each feasible pairing, attaches structured explanations, and returns a ranked list.

It reads snapshots by identifier only; it never mutates candidate or job stores. It may fuse multiple retrieval signals and optional reranking stages when configured, but the default portal path favors a fixed composite strategy whose components users can inspect (§4, §5). When profile or job events indicate that inputs changed, the matchmaking layer drops cached rankings for the affected entity and recomputes on the next request.

The Matchmaking Agent suggests; it does not hire, reject, or apply on anyone’s behalf.

3.4 Agent Communication and State Sharing

JobMatch relies on exactly three cooperating agents: the **Candidate/Client Agent**, the **Employer Agent**, and the **Matchmaking Agent** (Figures 1 and 5). Sections 3.1–3.3 describe each agent’s role; this subsection explains how they exchange information, what each one is allowed to keep private, and how user actions propagate through the system without collapsing ownership boundaries.

What each agent owns.. The Candidate/Client Agent owns everything derived from the job seeker’s side of the market: the resume-derived profile, stated preferences (salary, remote work, location constraints), normalized skills, and the embedding-backed *candidate state* used for comparison. The Employer Agent owns the parallel objects on the hiring side: the job-description-derived posting, structured requirements, compensation and experience bounds, and the *job state* with its embedding. The Matchmaking Agent owns neither profile store. It owns the retrieval-and-ranking workflow—how candidate and job snapshots are paired, scored, explained, and ordered—and the transient recommendation output returned to a portal session. It may cache recent rankings for responsiveness, but it does not author candidate or job records.

State each agent maintains.. On the candidate and employer sides, each agent maintains two related layers. A **profile** is the editable portal record: form fields the user can revise at any time. A **snapshot** is a versioned, match-ready view frozen at the moment the user confirms a save—normalized skills, constraint values, document identity, and a comparable text representation with its embedding. Only snapshots enter ranking; profiles exist so people can edit without immediately rewriting history. The Matchmaking Agent maintains session-scoped ranking state (cached lists, last-computed scores) and structured explanation payloads attached to each recommendation. It does not maintain a third copy of full resumes or job descriptions beyond what it needs to score a request.

What data is shared.. Agents share the minimum information required for fair comparison and timely refresh. **Snapshots and identifiers** cross the boundary to the Matchmaking Agent when a user requests matches: a candidate identifier plus the

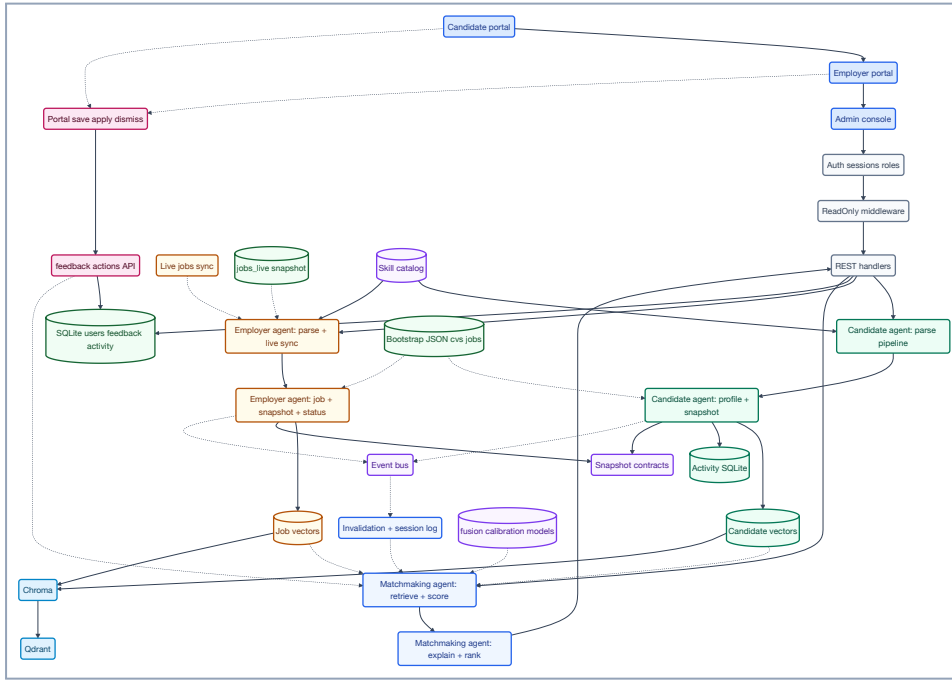


Fig. 5: End-to-end block architecture: three portals, API gateway with auth and read-only middleware, three agents (abbreviated from Figures 2-4), persistence layer (SQLite, bootstrap JSON, vectors), shared skill catalog and event bus, feedback loop, and vector backends (Chroma/Qdrant).

lists with explanations; they are shared with the UI layer, not written back into profile stores.

What should not be shared unnecessarily.. The design withholds data that would blur accountability or expose one party without cause. The Candidate Agent does not receive employer-only draft postings or internal recruiter notes unless the employer publishes a job snapshot. The Employer Agent does not receive another candidate’s full profile when an employer views a shortlist—only the fields appropriate to that match request. The Matchmaking Agent never receives portal credentials, authentication secrets, or raw feedback logs as inputs to ranking; save, dismiss, and apply actions are recorded separately and do not silently rewrite embeddings. Neither owning agent calls the other’s write path: all cross-side access for matching is read-only through the Matchmaking Agent.

How vector stores are updated.. Each owning agent updates only its own embedding index when a confirmed snapshot changes. After parsing and normalization, the agent computes a fresh vector for the canonical document text and upserts it into the candidate or job vector store. The opposite side’s store is untouched. An update event follows the write so downstream components know prior similarity searches may be outdated. The Matchmaking Agent never performs upserts; it queries both stores

when retrieving candidates for a job or jobs for a candidate. Implementation details of the store backends appear in §4.

How the Matchmaking Agent reads from both sides.. A match request supplies direction (candidate seeking jobs, or employer seeking candidates), the authenticated user’s snapshot identifier, and retrieval scope (top- K , strategy). The agent loads the query-side snapshot from the owning agent’s store, retrieves candidate vectors or job vectors from the respective indexes, scores each feasible pair using the configured strategy, attaches explanations, and sorts the results. Reads are symmetric: the same pipeline runs in reverse when an employer searches for candidates. If either store lacks a fresh embedding, the owning agent must register a snapshot first; the Matchmaking Agent does not fabricate missing state.

How UI actions trigger agent updates.. Portal interactions map to owning agents and to the Matchmaking layer. When a candidate uploads or edits a resume and confirms the form, the Candidate Agent updates profile and snapshot state, refreshes the candidate embedding, and emits a profile-updated event. When an employer posts or revises a job description and saves, the Employer Agent performs the parallel job-side update. Requesting the matches page—or an employer opening reverse search for a posting—invokes the Matchmaking Agent with the current snapshots; no ranking runs on half-edited forms until the user confirms. Secondary actions such as save, apply, shortlist, or dismiss are recorded as user activity; they do not automatically change embeddings unless the user later edits and re-saves a profile or job. Administrators can inspect recent events for debugging but do not override ownership through the UI.

Human-in-the-loop by design.. This communication model keeps people in control of consequential outcomes. Agents prepare, normalize, and suggest; they do not substitute for a hiring decision. Ranked lists arrive with reasons attached, and every consequential action—saving a job, submitting an application, contacting a candidate—requires an explicit portal choice. Because snapshots change only after user confirmation, a person can review parsed fields, correct errors, and understand what the system will compare before any recommendation is computed. Separation of ownership plus read-only matchmaking makes it clear who changed what: candidates govern candidate state, employers govern job state, and the Matchmaking Agent governs only how those states are compared at a given moment.

3.5 Overall Workflow

Figure 6 traces the candidate path; Figure 7 traces the employer path. In both portal workflows, document ingestion is assistive: extracted fields populate review forms, users confirm or correct them, and only then does the owning agent register a snapshot and notify the matchmaking layer.

Algorithm 1: Multi-Agent Semantic Job Matching

Input: Candidate resumes or CVs; job descriptions; candidate preferences when available; employer constraints when available.

Output: Ranked candidate–job matches; matching scores; structured explanations and matched-skill summaries.

1. **Candidate Agent** receives a resume or CV from the candidate portal.
2. **Candidate Agent** preprocesses and parses the document into assistive form fields.

3. **Candidate Agent** extracts skills, experience, education, and stated preferences; the user confirms or edits the result.
4. **Candidate Agent** generates a candidate embedding from the confirmed canonical text.
5. **Candidate Agent** stores the candidate profile, versioned snapshot, and vector representation; emit a profile-updated event.
6. **Employer Agent** receives a job description from the employer portal.
7. **Employer Agent** preprocesses and parses the job description.
8. **Employer Agent** extracts role title, required skills, experience bounds, compensation band, location or remote policy, and hiring constraints; the user confirms or edits the result.
9. **Employer Agent** generates a job embedding from the confirmed canonical text.
10. **Employer Agent** stores the job profile, versioned snapshot, and vector representation; emit a job-updated event.
11. **Matchmaking Agent** retrieves candidate and job representations from both vector stores (read-only).
12. **Matchmaking Agent** computes semantic similarity and optional constraint-aware component scores (formulas in §5).
13. **Matchmaking Agent** ranks candidate–job pairs by the configured strategy (portal default: composite score).
14. **Matchmaking Agent** generates an explanation and matched-skill summary for each ranked pair.
15. **UI/Application Layer** displays ranked recommendations; the user explicitly chooses save, apply, shortlist, or contact.

Algorithm 1 summarizes the end-to-end multi-agent matching process at the architectural level; score definitions appear in §5 and implementation paths in §4.

Candidates browse ranked jobs with explanation panels, then explicitly save or apply. Employers browse ranked candidates for a posting, then shortlist or contact through portal actions. Administrators inspect agent health and recent events in a separate console. Separate web experiences for candidates, employers, and administrators sit above the gateway; routing, endpoints, and default request parameters are specified in §4.

4 Implementation

This section describes how the architecture in §3 is realized in the repository: gateway routes, agent modules, stores, portal surfaces, typed contracts, parameter defaults, and offline evaluation assets. Figure 8 details the matching pipeline from snapshot inputs through retrieval, composite scoring, constraints, optional branches, and portal-facing explanations. Score definitions appear in §5; measured outcomes in §6.

4.1 Backend implementation

Gateway and route map. The HTTP entry point is `backend/gateway/app.py`: a FastAPI application (`build_gateway`) wired to a `SystemContainer` from `bootstrap.py`. Routers mount at the application root (no global `/api` prefix):

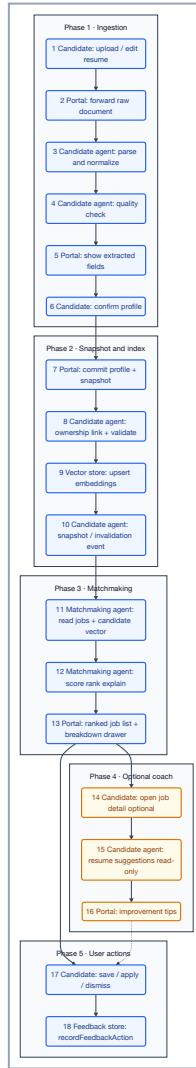


Fig. 6: Candidate workflow: resume upload, assisted parsing, quality check and form prefill, profile confirmation with ownership link, snapshot registration, vector upsert, ranked job discovery with explanation drawer, and explicit save/apply/dismiss actions recorded to the feedback store.

`/auth` (`auth/routes.py`), `/agents` (`gateway/routes/agents.py`), `/candidates` (`gateway/routes/candidates.py`), `/jobs` (`gateway/routes/employers.py`), `/match/*` and legacy aliases (`gateway/routes/matching.py`), `/similar` (`gateway/routes/similar.py`), `/feedback` (`gateway/routes/feedback.py`), `/system` (`gateway/routes/system.py`), `/real-jobs` (`gateway/routes/real_jobs.py`). Middleware: `SessionMiddleware`

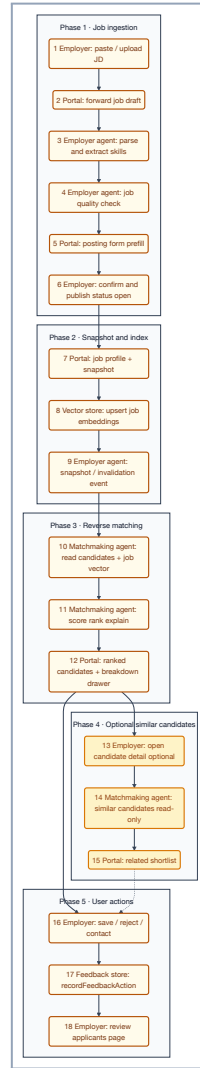


Fig. 7: Employer workflow: JD ingestion, job quality check, posting confirmation with status open, snapshot registration, reverse candidate matching with explanation drawer, feedback actions (save/reject/contact), and applicants page review.

(cookie `jm_session`, 7-day max age) and optional `ReadOnlyMiddleware` when `read_only=true`. Default listen address: `0.0.0.0:8001` (`config.py`).

Agent modules.. Three domain agents are constructed in `bootstrap.py` and share an in-process `AgentEventBus` (`bus/event_bus.py`): **CandidateAgent** (`agents/candidate_agent.py`) owns candidate profile ingestion, `JsonParser` resume parsing (`hooks/parser.py`), snapshot versioning, and vector upserts

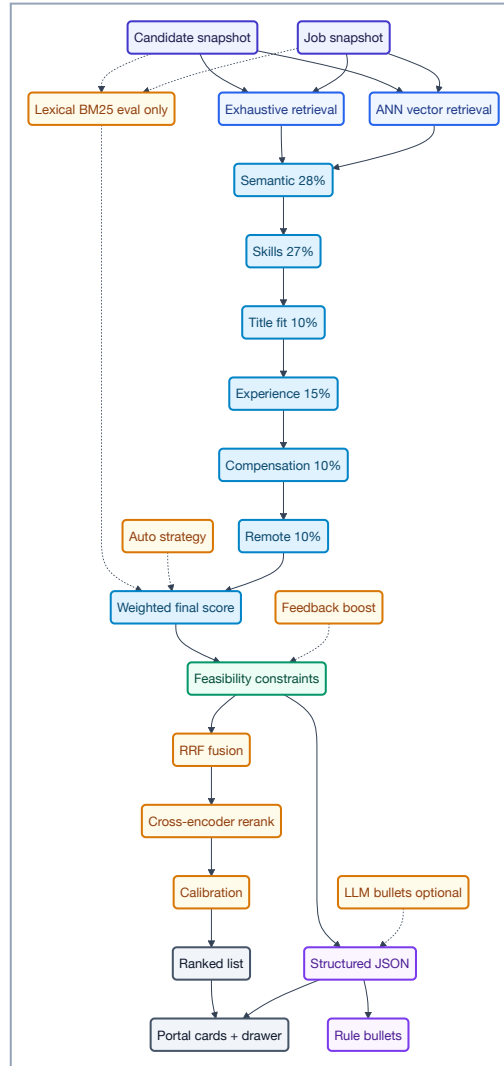


Fig. 8: Implementation of the matching pipeline: retrieval (exhaustive or ANN primary; lexical benchmark-only), composite scoring with six weighted components, feasibility constraints, optional fusion/feedback boost/auto-strategy, and structured explanation fork (rule default, LLM optional) consumed by portal cards and drawer.

to `candidates_collection`; **EmployerAgent** (`agents/employer_agent.py`) mirrors this for job postings in `jobs_collection`, including `replace_corpus` for bulk refresh from external feeds or snapshots; **MatchmakingAgent** (`agents/matchmaking_agent.py`) is read-only with respect to profile stores: it loads snapshots, runs retrieval and scoring, builds **MatchResult** lists, and publishes

MATCH_COMPLETED events. Each agent extends `BaseAgent` (`agents/base.py`) for status reporting exposed at `GET /agents/status`.

Matching core.. Scoring and ranking live under `backend/core/`: `scoring.py` (composite, multimodal, semantic-only, skill-only strategies; `COMPOSITE_WEIGHTS`), `matchmaking_scoring.py` (pair scoring orchestration, optional constraints, calibration, feedback boost), `similarity.py`, `skills.py`, `component_scores.py`, `match_explanation.py` and `explain.py` (structured and rule-based explanations), `rrf.py` (ensemble list fusion), `fusion.py` (learned logistic combiner), `calibration.py` (Platt scaling), `feedback_boost.py`, `constraints.py`, `strategy_router.py`, `cross_encoder_rerank.py`, `lexical.py` (BM25/TF-IDF; offline benchmarks only). Embeddings are computed in `core/embedding.py` via Sentence-Transformers.

Stores.. **Vector stores:** `stores/factory.py` selects `ChromaVectorStore` (default, `vector_store=chroma`, persist path `backend/chroma_db/`) or `QdrantVectorStore` (`vector_store=qdrant`, `backend/qdrant_db/`) behind a common upsert/query interface; cosine space is the Chroma default. **SQLite** (`backend/app.db`, path from `Settings.sqlite_path`): `auth/store.py` (`UserStore`, roles and sessions), `stores/feedback_store.py` (match and user feedback rows), `stores/candidate_activity_store.py` (saved jobs, applications, activity log). **Frozen JSON corpus:** `data/cvs.json`, `data/jobs.json` seed demo accounts when `seed_demo=true`.

External live jobs integration.. `core/real_jobs_sync.py` implements the upstream contract: paginated `GET` requests with `limit/skip`, field normalization (`normalize_external_job`), identifier deduplication, and snapshot persistence to `data/jobs_live.json`. `services/real_jobs_service.py` orchestrates sync at runtime: fetch upstream, write snapshot, call `EmployerAgent.replace_corpus`, and optionally re-embed candidates and jobs (`reindex_all`). Gateway endpoints: `GET /real-jobs/status` (configuration and last sync state) and `POST /real-jobs/sync` with body `{reindex: bool}` (default `true`). On startup, `bootstrap.py` loads `jobs.json`, then replaces the employer corpus from `jobs_live.json` when that snapshot contains jobs. Environment gates: `REAL_JOBS_ENABLE`, `REAL_JOBS_BASE_URL`, `REAL_JOBS_PATH`, `REAL_JOBS_PAGE_LIMIT`, `REAL_JOBS_TIMEOUT_SEC`, `REAL_JOBS_OUTPUT_PATH` (`backend/.env.example`). CLI one-shot sync: `python backend/scripts/sync_real_jobs_once.py`. Daily batch requests may set `sync_before_run=true` (default) to pre-sync when live jobs are enabled.

Authentication and role separation.. Roles are `candidate`, `employer`, and `admin` (`auth/deps.py`, `require_role`). Session auth endpoints: `POST /auth/register`, `POST /auth/login`, `POST /auth/logout`, `GET /auth/me`. Role-gated routes enforce ownership at the gateway layer, e.g. `/candidates/me/*` and resume upload (`candidate`), `/jobs/mine/*` and description upload (`employer`), `/similar/jobs/*` (`candidate`), `/similar/candidates/*` (`employer`), `POST /system/demo/reset` (`admin`). Matching endpoints (`/match/*`) and agent telemetry (`/agents/*`) are callable without role checks so the admin console and integration tests can invoke them directly; production portals still authenticate users before calling profile-scoped routes.

Table 1 summarizes end-to-end flows; representative endpoint groups follow.

Candidate API surface.. GET/PUT /candidates/me, POST /candidates/upload-resume, POST /candidates/me/resume-suggestions, POST /candidates/quality-check, saved jobs (GET/PUT /candidates/me/saved-jobs), applications (GET/POST /candidates/me/applications).

Employer API surface.. GET/POST /jobs, PUT /jobs/mine/{job_id}, PATCH /jobs/mine/{job_id}/status, description upload/parse (POST /jobs/upload-description, POST /jobs/parse-description), GET /jobs/mine/applications.

Matching API surface.. POST /match/candidate-to-jobs, POST /match/job-to-candidates, POST /match/ensemble, POST /match/daily-batch (optional pre-sync via `sync_before_run`), plus legacy aliases /match-resume, /match-job, /match-resume-ensemble, /agent/run-daily-recommendations.

External jobs API surface.. GET /real-jobs/status, POST /real-jobs/sync.

Feedback, similar items, and system routes.. POST /feedback, POST /feedback/actions, GET /feedback/me, GET /feedback/counts; GET /similar/jobs/{job_id}, GET /similar/candidates/{candidate_id}; GET /system/config, GET /system/fairness, POST /system/vector-store, POST /system/demo/reset (when demo mode enabled). Table 1 summarizes authenticated portal routes, user goals, and the gateway paths that implement them.

4.2 Frontend implementation

The UI is a single React 19 SPA (Vite build, `frontend/`) with axios session cookies and role-guarded routes in `App.jsx` (`ProtectedRoute`).

Candidate portal (role=candidate).. Routes: /candidate/onboarding, /candidate/profile, /candidate/matches, /candidate/saved. Pages call `frontend/src/api/client.js` with `DEFAULT_CANDIDATE_MATCH` (see §4.5). Profile save dispatches a client-side `jobmatch:profile-updated` event (`utils/profileEvents.js`); matches and saved views refetch on that signal. Explainability UI: `MatchExplainability`, `MatchDetailsDrawer`; users can save jobs, dismiss matches, and submit applications explicitly.

Employer portal (role=employer).. Routes: /employer/jobs, /employer/matches, /employer/applications. Job editor and description upload feed /jobs/mine/* endpoints; reverse matching uses `DEFAULT_EMPLOYER_MATCH` with `mode=job_to_candidates`. Applicant lists read GET /jobs/mine/applications.

Admin and evaluation view (role=admin).. Route: /admin/console (`pages/admin/AdminConsole.jsx`). Panels: `AgentStatusPanel`, `AgentEventStrip` (tails GET /agents/events/recent), `SystemConfigPanel`, `MatchControls` (manual `runMatch` / `runDailyBatch`), `ResultsPanel`, and fairness summary from GET /system/fairness. Recent run metadata is cached in `localStorage` (`jm_recent_runs`). Live job sync is invoked via API or CLI (§4); a dedicated admin UI control is not yet wired.

Shared layout and error handling.. All portals render inside `PortalShell` (`layouts/PortalShell.jsx`) with role-aware navigation. Dedicated error routes (`pages/errors/ErrorMessage.jsx`) present full-screen, centered recovery pages for

Table 1: Primary user-facing workflows by role ($K=10$ composite matching unless noted).

Role	Portal route	User goal	API / agent path
Candidate	<code>/candidate/onboarding</code>	Upload resume, review extraction, create profile	POST <code>/candidates/upload-resume</code> → PUT <code>/candidates/me</code> → Candidate agent registers snapshot + embedding
Candidate	<code>/candidate/profile</code>	Edit skills, experience, pay, remote prefs	GET/PUT <code>/candidates/me</code> ; quality via POST <code>/candidates/quality-check</code>
Candidate	<code>/candidate/matches</code>	View ranked jobs, filter, explain scores	POST <code>/match/candidate-to-jobs</code> (composite); profile readiness gate before search
Candidate	<code>/candidate/saved</code>	Saved jobs and applications	GET/PUT <code>/candidates/me/saved-jobs</code> ; POST <code>/candidates/me/applications</code>
Employer	<code>/employer/jobs</code>	Import JD, post/edit/close roles	POST <code>/jobs/upload-description</code> or <code>/parse-description</code> → POST <code>/jobs</code> or PUT <code>/jobs/mine/{id}</code> → Employer agent
Employer	<code>/employer/matches</code>	Rank candidates for an owned job	POST <code>/match/job-to-candidates</code> (composite); job ownership required
Employer	<code>/employer/applications</code>	Review applicants per posting	GET <code>/jobs/mine/applications</code> (SQLite activity store)
Admin	<code>/admin/console</code>	Monitor agents, run matches, inspect fairness	GET <code>/agents/status</code> , <code>/agents/events/recent</code> , <code>/system/config</code> ; optional <code>/match/*</code> , <code>/system/fairness</code> , POST <code>/real-jobs/sync</code> when live feed enabled

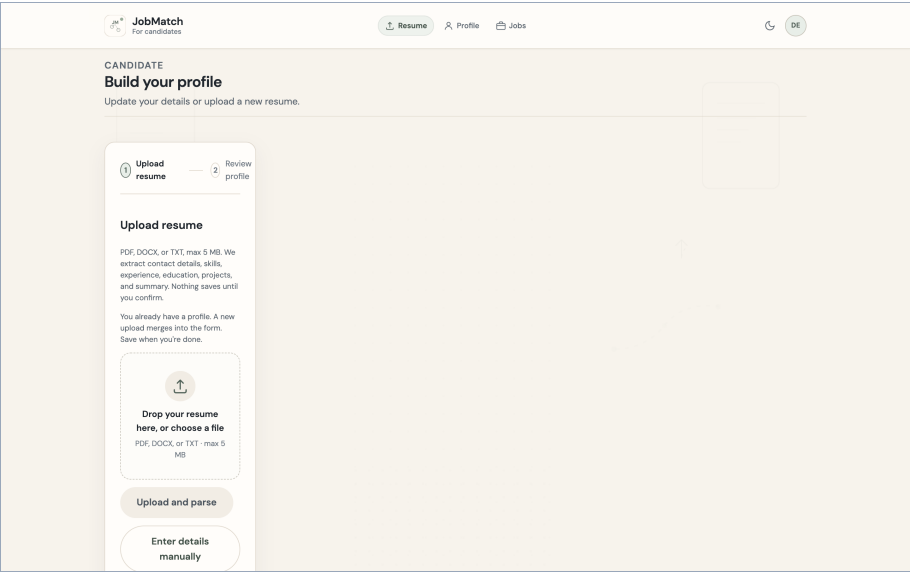
All authenticated routes use cookie sessions (`withCredentials`) and FastAPI `require_role` guards. Match endpoints invoke the Matchmaking agent read-only over snapshots.

HTTP 404/502 and unknown failures, with copy that distinguishes client-side routing errors from upstream outages.

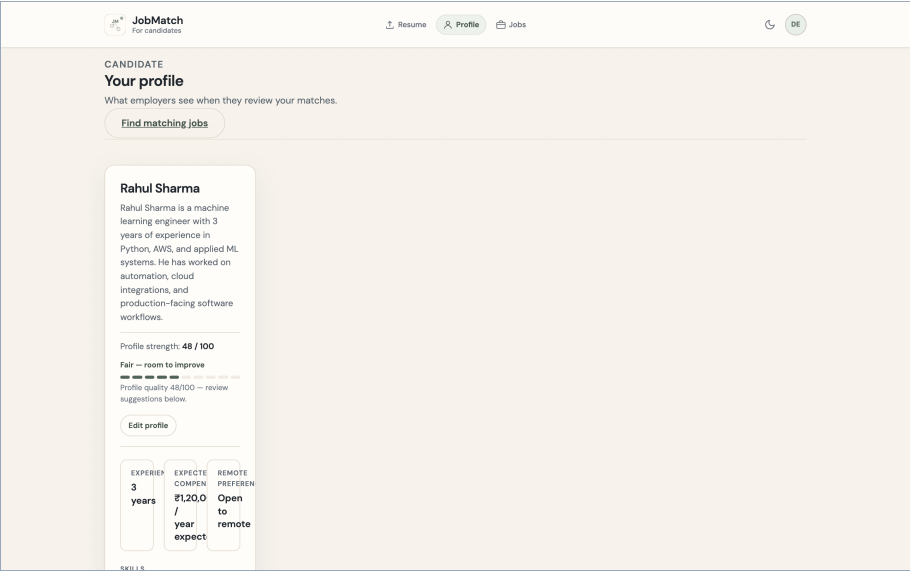
Figure 9 shows representative portal surfaces across candidate onboarding, profile editing, bidirectional matching, employer job management, and the admin research console that invokes the matching pipeline in Figure 8. Screenshots use the demo corpus and default composite strategy where applicable (§4.5); they illustrate the research prototype UI rather than usability study outcomes.

4.3 Data structures

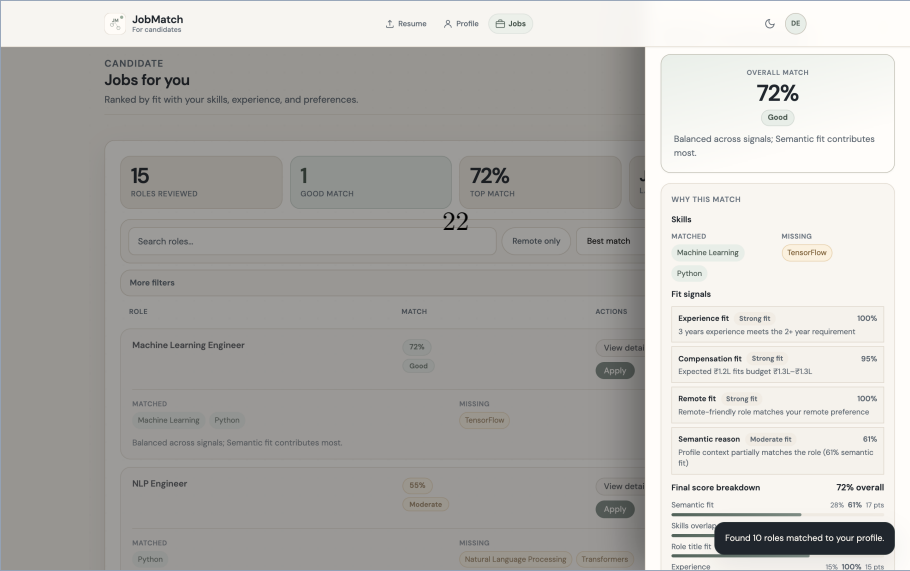
Contracts are Pydantic models in `backend/contracts/`; snapshots are the only objects the Matchmaking agent scores.



(a) Candidate resume onboarding



(b) Candidate profile editor



(c) Candidate job matches

Candidate profile and CandidateSnapshot.. Editable portal fields: name, skills, experience years, preferred salary, remote preference, summary, canonical resume text. On save, `CandidateSnapshot` (`contracts/snapshots.py`) stores `id`, `name`, `skills`, `experience_years`, `remote_preference`, `preferred_salary`, `summary`, `monotonic_version`, `document_text_hash`, and fixed-length `embedding`.

Job profile and JobSnapshot.. Posting fields: title, description, `required_skills`, `preferred_skills`, `required_experience`, `budget` (`budget`, `budget_min`, `budget_max`), `remote_policy`, optional company, link, location, `job_type`, source, and `posted_at` when supplied by portal or external feeds, plus the same `version`, `document_text_hash`, and `embedding` triple as candidates. Vector metadata upserts store company, link, location, compensation bounds, and source for portal filters and admin inspection.

MatchResult and MatchExplanation.. `MatchResult` (`contracts/matching.py`) carries `target_id`, `target_label`, `rank`, bi-encoder similarity, per-component scores (`semantic_score`, `skills_score`, `title_score`, `experience_score`, `compensation_score`, `remote_score`), `final_score`, skill overlap lists, optional contact and job metadata, constraint notes, and an optional `MatchExplanation`. `MatchExplanation` bundles `matched_skills`, `missing_skills`, four `FitSignal` rows (`semantic`, `experience`, `compensation`, `remote`), and `score_breakdown` (`ScoreComponentDetail` list with weight, score, contribution). The enclosing `MatchResponse` adds `session_id`, `direction`, `strategy_used`, corpus counters, agent version map, and optional rerank diagnostics.

Feedback and agent events.. **Feedback:** `FeedbackStore` rows record `user_id`, `target_id`, `action`, optional `context_id`, `role`, and timestamp. Match-scoped actions: `save`, `dismiss`, `apply`; broader user actions: `contact`, `reject`, `not_interested`. Feedback does not rewrite embeddings; it can adjust scores only when `use_feedback_boost=true`. **Agent events:** typed enums in `bus/events.py`—`CANDIDATE_PROFILE_UPDATED`, `JOB_PROFILE_UPDATED`, `CORPUS_BOOTSTRAPPED`, `MATCH_REQUESTED`, `MATCH_COMPLETED`—published on the in-process bus and surfaced to the admin event strip.

4.4 Libraries and frameworks

Only dependencies declared in the repository are listed here.

Backend (`backend/requirements-min.txt`).. FastAPI 0.129, Uvicorn 0.29, Pydantic 2.9, `pydantic-settings`, NumPy 1.26, Sentence-Transformers 5.1, ChromaDB 0.4.24, HTTPX, Passlib/bcrypt, `python-multipart`, `python-dotenv`, pdfplumber, `email-validator`, `ItsDangerous`; optional Qdrant client 1.12 and tiktoken 0.8 when those backends are enabled. SQLite access uses the Python standard library (`sqlite3`).

Frontend (`frontend/package.json`).. React 19, React DOM, React Router 7, axios 1.7; dev/build tooling Vite 6 with `@vitejs/plugin-react`.

Monolith layout.. JobMatch ships as one deployable backend process plus one static SPA bundle; there is no inter-service RPC layer beyond the in-process event bus.

4.5 Parameter settings

Embedding model and vector backend.. Default embedding model: `all-MiniLM-L6-v2` (384 dimensions, `Settings.embedding_model`). Vector backend selector: `Settings.vector_store` $\in \{\text{chroma}, \text{qdrant}\}$; Chroma is the default in committed settings.

Top- K and retrieval.. Portal defaults (`DEFAULT_CANDIDATE_MATCH`, `DEFAULT_EMPLOYER_MATCH` in `client.js`): `topK=10`, `retrieval=exhaustive` on the demo corpus (15 jobs / 30 resumes). API contract defaults (`MatchRequest`): `top_k=5`, `candidate_pool=120`, `retrieval=exhaustive`; offline evaluation and regression tests use $K=5$ unless a driver overrides `--top-k`. When `retrieval=ann`, shortlists come from the configured vector store before pairwise rescoring.

Scoring weights and modes.. Portal requests fix `strategy=composite`, `metric=cosine`, `skillsMode=jaccard`, `semanticWeight=0.7`, `fusionMode=fixed`, with `applyConstraints=false`, `autoStrategy=false`, `useCalibration=false`, `useFeedbackBoost=false`, `explainMode=rules`. Composite component weights in `COMPOSITE_WEIGHTS`: semantic 28%, skills 27%, title 10%, experience 15%, compensation 10%, remote 10% (Eq. 6). RRF merge constant: `Settings.rrf_k=60`. Learned fusion weights load from `data/models/fusion.json`; Platt calibration from `data/models/calibration.json` ($a \approx 2.078$, $b \approx 2.112$).

Thresholds and optional boosts.. Explainability fit labels (`match_explanation.py`): Strong ≥ 0.85 , Good ≥ 0.65 , Moderate ≥ 0.45 , else Weak. Rule bullets in `explain.py` use additional cutoffs (e.g. title ≥ 0.65 , semantic narrative bands at 0.75 and 0.55). Hard constraint penalty default in `constraints.py`: factor 0.65 when experience gap triggers. Feedback boost deltas when enabled (`feedback_boost.py`): save +0.04, apply +0.06, dismiss -0.06. Cross-encoder rerank pool default: 20; disabled in production settings (`enable_cross_encoder_rerank=false`).

External live jobs (optional).. Gated by `REAL_JOBS_ENABLE=false` by default. When enabled: `REAL_JOBS_BASE_URL`, `REAL_JOBS_PATH` (default `/jobs`), `REAL_JOBS_PAGE_LIMIT` (100), `REAL_JOBS_TIMEOUT_SEC` (30), snapshot path `REAL_JOBS_OUTPUT_PATH` (`data/jobs_live.json`). Daily batch default `sync_before_run=true` pre-syncs only when live jobs are enabled.

Vector-store update path.. On snapshot registration the owning agent (1) embeds canonical document text, (2) upserts into the active vector collection, and (3) emits a profile-updated event. The Matchmaking agent reads snapshots, scores all or ANN-filtered pairs with the requested strategy, attaches explanations, and returns the top- K list.

4.6 Reproducibility

Dataset files.. Primary evaluation corpus under `data/`: `cvs.json` (30 resumes), `jobs.json` (15 jobs), `eval_pairs.json` (47 graded query-document pairs with relevance labels). Auxiliary assets: `fairness_audit_profiles.json`, frozen expected metrics `expected/paper_progression_summary.json`, trained auxiliary models

models/fusion.json and models/calibration.json. Research export mirror: data/research/ (cvs.json, jobs.json, eval_pairs.json, manifest.json).

Benchmark drivers.. Python modules under backend/benchmarks/ share eval_data.py snapshot builders and metrics.py rank metrics. Entry scripts include run_eval.py, run_comparison.py, run_ablation.py, run_paper_tables.py, run_significance.py, run_explainability_eval.py, run_fairness_audit.py, run_generate_research_dataset.py, and orchestrator research_pipeline.py. One-shot wrapper: backend/scripts/run_research_pipeline.py (default --top-k 5).

Automated tests.. Layout: tests/unit/backend/ (Python unit tests), tests/unit/frontend/ (Node tests for frontend/src/Utils/*), tests/integration/ (HTTP flows), tests/benchmarks/ (offline eval regression). Unified runner from repository root: bash scripts/run_tests.sh (pytest ../tests plus Node utils). Regression gate: tests/benchmarks/test_eval_regression.py recomputes semantic and multimodal rankings on the frozen corpus and compares nDCG@5 against paper_progression_summary.json with tolerance ± 0.04 (Eq. 14). Live-jobs contract tests: tests/unit/backend/test_real_jobs_sync.py (normalize, pagination mock, snapshot I/O) and tests/integration/test_real_jobs_api.py (status endpoint; sync disabled returns HTTP 400). Additional benchmark tests cover ablations, fusion tables, fairness audit, explainability checks, significance resampling, synthetic dataset validation, and the research export pipeline (tests/benchmarks/*.py). Run backend tests from backend/ with pytest ../tests -q (310 tests collected as of submission tag; declared in requirements-min.txt).

TODO: Commit phase11_summary.csv from benchmarks/phase11.py for ANN shortlist latency.

5 Quality Metrics

This section defines the scores computed inside the Matchmaking agent and the ranking, explanation, and fairness criteria used in offline evaluation. Figure 10 summarizes the evaluation pipeline from the frozen demo corpus through benchmark drivers, metrics, audits, and regression gates. Implementation modules appear in §4; numeric outcomes appear in §6.

5.1 Evaluation corpus and protocol

Offline evaluation treats each resume as a query and each job posting as a candidate document. The labeled set comprises 30 resumes, 15 jobs, and 47 graded query-document pairs in data/eval_pairs.json. Relevance grades are $r \in \{0, 1, 2\}$ (none, partial, strong); queries with $r > 0$ form the relevant set for binary metrics, while nDCG uses the full graded scale. Table 2 summarizes corpus statistics. Snapshots are built with the same embedding path used in production (§4).

Unless a driver overrides `top_k`, we report macro-averaged metrics at $K=5$ over all 30 queries. Each benchmark driver calls the shared `eval_rankings()` routine

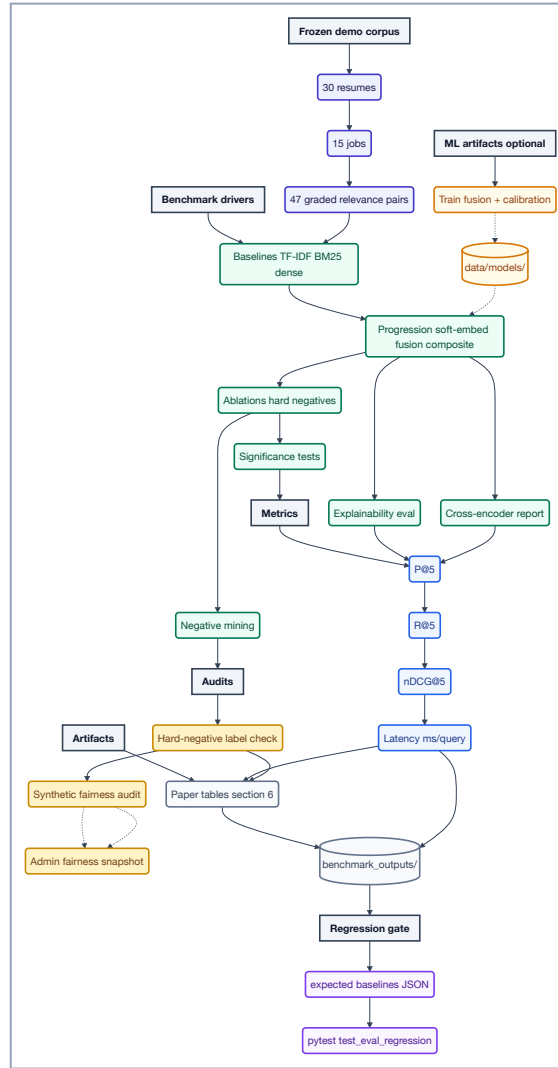


Fig. 10: Quality evaluation pipeline: frozen demo corpus, benchmark drivers (base-lines, progression, ablations, significance, explainability, cross-encoder report, negative mining), metric aggregation, fairness audits, optional ML training of fusion/calibration artifacts, regression gates, and paper tables. Evaluation runs via CLI/pytest only.

(`benchmarks/metrics.py`) so that method comparisons differ only in the ranking function, not in metric code.

Table 2: Evaluation corpus statistics.

Statistic	Value
Task	Resume $\rightarrow$ jobs (candidate-initiated retrieval)
Candidates (queries)	30
Job postings (corpus)	15
Graded query-document pairs	47
Queries with ≥ 1 relevant job	30
Relevance scale	0 (none), 1 (partial), 2 (strong)
Grade-2 (strong) labels	21
Grade-1 (partial) labels	26
Labels per query (min / max)	1 / 2
Embedding model	<code>all-MiniLM-L6-v2</code> ($d=384$)
Label source	<code>data/eval_pairs.json</code> v1.0

Starter labeled set for offline benchmarks; not derived from live portal traffic.

5.2 Matching scores

All component scores are normalized to $[0, 1]$. Let $\mathbf{e}_c, \mathbf{e}_j \in \mathbb{R}^d$ denote candidate and job snapshot embeddings ($d=384$), and let A and B be canonical skill sets on the resume and required job skills respectively.

Semantic similarity.. The semantic score measures how closely resume and job-description embeddings align. Cosine mode (portal default):

$$s_{\text{sem}}^{\text{cos}} = \frac{\mathbf{e}_c \cdot \mathbf{e}_j}{\|\mathbf{e}_c\| \|\mathbf{e}_j\|}. \quad (1)$$

Euclidean mode maps distance to similarity:

$$s_{\text{sem}}^{\text{euc}} = \frac{1}{1 + \|\mathbf{e}_c - \mathbf{e}_j\|_2}. \quad (2)$$

Skill overlap.. The skill score measures overlap between resume skills and job requirements. Jaccard mode (portal default):

$$s_{\text{skl}}^{\text{jac}} = \begin{cases} |A \cap B| / |A \cup B| & \text{if } A \neq \emptyset \text{ and } B \neq \emptyset, \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

Embedding mode averages, for each required skill $b \in B$, the maximum cosine between b 's skill phrase embedding and any resume skill embedding:

$$s_{\text{skl}}^{\text{emb}} = \frac{1}{|B|} \sum_{b \in B} \max_{a \in A} \cos(\mathbf{e}_a^{(\text{skill})}, \mathbf{e}_b^{(\text{skill})}). \quad (4)$$

Multimodal weighted score (research baseline).. A two-signal blend combines semantic and skill scores with weight α (default $\alpha=0.7$):

$$s_{\text{mm}} = \alpha s_{\text{sem}} + (1 - \alpha) s_{\text{skl}}. \quad (5)$$

Composite score (portal default).. The composite score is a fixed weighted sum over six interpretable components—semantic, skills, title token overlap, experience fit, compensation fit, and remote preference—then clipped to $[0, 1]$:

$$s_{\text{final}} = \text{clip}_{[0,1]} \left(\sum_k w_k s_k \right), \quad (w_{\text{sem}}, w_{\text{skl}}, w_{\text{tit}}, w_{\text{exp}}, w_{\text{comp}}, w_{\text{rem}}) = (0.28, 0.27, 0.10, 0.15, 0.10, 0.10), \quad (6)$$

where s_{tit} , s_{exp} , s_{comp} , and s_{rem} are rule-based scalars in `component_scores.py` (piece-wise mappings from experience gap, budget band, and remote-policy fields). Title overlap blends token coverage and Jaccard overlap between job-title tokens and resume summary/skills text.

Ensemble and optional adjustments.. Reciprocal rank fusion (RRF) merges m ranked lists without retraining:

$$\text{RRF}(x) = \sum_{i=1}^m \sum_r \mathbb{I}[\mathcal{L}_i[r] = x] \cdot \frac{1}{k + r}, \quad (7)$$

with smoothing constant $k=60$ by default. When enabled, hard constraints multiply s_{final} by a feasibility factor $\phi \in (0, 1]$; Platt calibration maps scores to $\hat{p}(s) = \sigma(as + b)$ with fitted (a, b) ; feedback boost adds fixed save/apply/dismiss deltas. These optional paths are disabled in default portal requests (§4.5). *TODO*: Report calibration error (ECE or Brier) on held-out eval pairs. *TODO*: Report feedback-boost and calibration-on vs. calibration-off deltas on the eval corpus.

5.3 Ranking metrics

Given ranked list π_q for query q , graded relevance map $\text{rel}_q(\cdot)$, and relevant set $\mathcal{R}_q = \{j : \text{rel}_q(j) > 0\}$, let $\pi_q[:K]$ denote the top- K prefix.

Precision@K.. Precision@K is the fraction of the top- K slots occupied by relevant jobs:

$$\text{P@K}(q) = \frac{1}{K} \sum_{j \in \pi_q[:K]} \mathbb{I}[j \in \mathcal{R}_q]. \quad (8)$$

Recall@K.. Recall@K is the fraction of all relevant jobs for the query retrieved within the top- K :

$$\text{R@K}(q) = \frac{1}{|\mathcal{R}_q|} \sum_{j \in \pi_q[:K]} \mathbb{I}[j \in \mathcal{R}_q]. \quad (9)$$

nDCG@K.. Normalized discounted cumulative gain rewards higher grades at higher ranks. For graded relevance:

$$\text{DCG}@K(q) = \sum_{i=1}^K \frac{2^{\text{rel}_q(j_i)} - 1}{\log_2(i + 1)}, \quad (10)$$

$$\text{nDCG}@K(q) = \frac{\text{DCG}@K(q)}{\text{DCG}@K(q^*)}, \quad (11)$$

where π_{q^*} is the ideal ranking that sorts jobs by rel_q descending.

Mean reciprocal rank (MRR).. MRR treats relevance as binary ($r > 0$) and measures how early the first relevant job appears:

$$\text{RR}(q) = \begin{cases} 1/\text{rank}_q(j^*) & \text{if a relevant job } j^* \text{ appears at rank } \text{rank}_q(j^*), \\ 0 & \text{otherwise.} \end{cases} \quad (12)$$

Macro-averaged MRR = $\frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \text{RR}(q)$.

Mean average precision (MAP).. MAP averages precision at each relevant hit, normalized by $|\mathcal{R}_q|$:

$$\text{AP}(q) = \frac{1}{|\mathcal{R}_q|} \sum_{r=1}^{|\pi_q|} \text{P}@r(q) \cdot \mathbb{1}[\pi_q[r] \in \mathcal{R}_q], \quad (13)$$

where $\text{P}@r(q)$ is precision computed over the length- r prefix. Drivers report macro-averaged P@K, R@K, MRR, nDCG@K, and MAP unless noted otherwise.

5.4 Benchmark comparison criteria

Compared method families.. The research pipeline (`benchmarks/comparison.py`, `paper_progression.py`, `ablation.py`, `table11_fusion.py`) evaluates, on the *same* labeled corpus: (i) lexical baselines (TF-IDF cosine, BM25, exact skill overlap); (ii) dense strategies (semantic cosine/euclidean, skills Jaccard/embedding, multimodal weighted); (iii) composite and component ablations; (iv) fusion variants (RRF, learned logistic regression, hierarchical blending); (v) optional cross-encoder reranking and ANN shortlist sweeps (`phase11.py`). Primary comparison metrics are macro-averaged P@5, R@5, MRR, nDCG@5, and MAP at $K=5$.

Statistical comparison.. Paired bootstrap tests (`benchmarks/significance.py`) resample queries with replacement ($N=5000$, seed 42) and compare method pairs on nDCG@K and MRR; a difference is flagged significant at $\alpha=0.05$ when the two-sided bootstrap p -value falls below 0.05. *TODO*: Report significance tables for portal-default composite vs. best research baseline in §6.

Regression gate.. Continuous integration recomputes selected rankings and requires:

$$\left| \widehat{\text{nDCG}@5}_{\text{run}} - \text{nDCG}@5_{\text{expected}} \right| \leq 0.04, \quad (14)$$

against committed values in `data/expected/paper_progression_summary.json` (`tests/benchmarks/test_eval_regression.py`).

Hard-negative mining (sanity check).. From the top- $P=10$ multimodal pool per query, hard negatives are high-scoring jobs outside the labeled relevant set:

$$\mathcal{H}_q = \{j \in \pi_q[:P] : j \notin \mathcal{R}_q\}_{|\cdot| \leq 5}. \quad (15)$$

Success criterion: zero overlap between $\mathcal{H}_q$ and declared relevant jobs across all queries (label-set sanity, not ranking quality).

TODO: No committed `phase11` summary artifact at time of writing.

5.5 Explanation quality criteria

Explainability is evaluated offline on ranked pairs using `benchmarks/explainability_eval.py` and `explainability_checks.py`. Two explainer modes are compared: rule-based bullets (`explain_mode=rules`, portal default) and a grounded template explainer.

Faithfulness.. For each explanation, three hard checks must pass: (1) mention at least one matched or missing skill when such skills exist; (2) introduce no skill strings outside the candidate/job vocabulary; (3) align qualitative component claims (e.g. “high semantic similarity”) with the underlying `ScoreBreakdown`. Faithfulness is the fraction of hard checks passed (0–1).

Specificity.. Specificity scores how concrete the bullets are relative to generic-only templates (0–1 heuristic based on explicit skill references and parsed skill lists).

Consistency under profile variants.. For synthetic profile pairs, bullet-set Jaccard similarity and matched-skill equality are measured between explanations for two controlled variants of the same base profile. A pair is *consistent* when Jaccard ≥ 0.5 and matched-skill sets agree; drift is $1 - \text{Jaccard}$. An instance is *flagged* when any hard check fails.

TODO: Add explainability evaluation summary table to §6.

5.6 Fairness and sanity metrics

Fairness checks are proxy-based sanity audits on the synthetic demo corpus; they do not infer protected attributes from real users.

Proxy-group selection rates.. Queries are grouped by experience tier (junior/mid/senior) and remote preference (remote-seeker vs. onsite-flexible). For each group g , selection rate is the fraction of queries whose top- K average match score meets or exceeds the corpus median score (`core/fairness.py`). Disparate impact ratio (DIR) is $\min_g \text{rate}_g / \max_g \text{rate}_g$ (1.0 indicates parity). *TODO:* Persist `fairness_eval.py` proxy-group ratios to committed `benchmark_outputs/` artifacts.

Synthetic counterfactual audit.. Ten fabricated profile pairs in `fairness_audit_profiles.json` undergo controlled field edits (`benchmarks/fairness_audit.py`). Per pair and job, the audit records rank change, absolute score delta, and explanation drift (bullet-set Jaccard) between variants. A case is flagged when top- K rank shifts, score delta exceeds 0.01, explanation drift is

positive in the top- K union, or the top-1 job changes. Reported aggregates include flagged-pair count and maximum score shift; numeric outcomes appear in §6.

6 Results and Discussion

This section reports benchmark results on the demo corpus defined in §5, compares ranking methods, interprets outcomes for platform stakeholders, and discusses architectural implications together with measurement limits. All figures come from committed benchmark artifacts; they do not predict hiring outcomes at production scale.

6.1 Evaluation scope and protocol

We evaluated *resume-to-jobs matching*: for each of 30 sample resumes, the system ranks 15 job postings and we ask how often the right jobs appear near the top. Human annotators supplied 47 graded pairs—each pair links one resume to one job with a relevance score of none (0), partial (1), or strong (2). We report results at cutoff $K=5$, meaning we judge only the top five jobs shown to a user.

Four questions guided the evaluation. First, **precision**: among the five jobs shown, how many are actually relevant? Second, **recall**: of all relevant jobs for that person, how many appear in the top five? Third, **ranking quality (nDCG@5)**: do *strongly* relevant jobs appear above merely partial ones? Fourth, **speed**: how long does each ranking take on this small pool? Metrics are averaged across all 30 resumes so that no single profile dominates the score. Formal definitions appear in §5; the same scoring code powers both these offline tests and the live portals (§4).

6.2 Baseline methods

We compared methods commonly used in recruitment pipelines, alone and in combination.

Keyword-style methods.. TF-IDF and BM25 treat resumes and jobs as word lists; they reward shared terms and run very quickly. Exact skill overlap and Jaccard skill matching reward listed skills without understanding paraphrases.

Meaning-based methods.. Semantic cosine scoring compares embedding vectors of resume and job text so that related wording can match even when keywords differ. Multimodal methods blend semantic similarity with skill overlap (default blend: 70% semantic, 30% skills).

Combined and advanced methods.. Reciprocal rank fusion (RRF) merges several ranked lists. Learned fusion trains a small model to weight signals. Composite scoring—the portal default—adds title fit, experience, compensation, and remote preference on top of semantic and skill channels. We also tested optional cross-encoder reranking (a slower second-pass scorer) and post-hoc constraint filtering.

Table 3 summarizes the head-to-head comparison on the same labeled set. Multimodal weighted scoring achieves the highest ranking quality ($\text{nDCG@5} = 0.924$) among the methods listed, while BM25 and TF-IDF remain close ($\text{nDCG@5} \approx 0.89\text{--}0.90$). On this small job pool, lexical baselines remain competitive, so gains from

dense and hybrid matching are measurable but modest. Semantic similarity alone ($\text{nDCG@5} = 0.878$) underperforms both keyword baselines and multimodal blends, which supports combining *what the text means* with *which skills are named*.

Table 3: Retrieval method comparison (macro-averaged, $K=5$).

Method	Family	P@5	R@5	MRR	nDCG@5	MAP
Multimodal weighted	Embedding	0.287	0.933	0.961	0.924	0.867
RRF ensemble	Embedding	0.293	0.950	0.944	0.913	0.845
TF-IDF cosine	Lexical	0.293	0.950	0.918	0.898	0.842
BM25	Lexical	0.307	0.983	0.912	0.894	0.838
Semantic cosine	Embedding	0.267	0.867	0.931	0.878	0.810
Semantic euclidean	Embedding	0.267	0.867	0.931	0.878	0.810
Soft skill embed	Embedding	0.280	0.900	0.911	0.869	0.829
Exact skill overlap	Lexical	0.233	0.733	0.816	0.748	0.681
Skills Jaccard	Embedding	0.233	0.733	0.816	0.748	0.681

Source: research pipeline run 2026-05-27T150000Z/comparison/comparison_summary.json.
Exhaustive ranking over all 15 jobs per query. Latency in Table 8.

6.3 Main results

The progression study (Table 4) ranks every job for every resume and tracks how design choices accumulate. The best single configuration—multimodal scoring with soft skill embeddings—reaches $\text{nDCG@5} = 0.969$ and $\text{recall@5} = 1.00$, meaning every resume retrieves all of its labeled relevant jobs within the top five. Plain semantic cosine reaches $\text{nDCG@5} = 0.911$; adding richer text templates nudges it to 0.922 but still trails the best hybrid. Merging four lists with RRF (0.935) helps diversity but does not beat the strongest single hybrid on this corpus. Cross-encoder reranking improves over bare semantic search (0.939) yet still falls short of soft-embed multimodal (0.969) while adding far more compute (Table 8).

Table 5 asks whether learned or rule-based *fusion* of signals changes the story. Learned logistic-regression fusion ties the best progression run ($\text{nDCG@5} = 0.968$). Fixed multimodal blending and hierarchical skill weighting both score 0.924. Applying hard constraints *after* scoring lowers nDCG@5 to 0.908; filtering out imperfect fits can remove jobs that annotators marked as partially relevant, which hurts graded ranking scores even when recruiters might prefer stricter filters.

Table 6 records fitted parameters for optional calibration and learned fusion. These models are available in the codebase but turned off in default portal settings; they were trained on the same 47 labeled pairs used for evaluation, so their strong offline scores should be read as an exploratory ceiling, not a proven production upgrade.

Summary.. The main finding is modest but consistent: matching improves when the system combines language semantics with explicit skills, and the highest scores arise from hybrid signals rather than either channel alone. No method achieves high precision—at best, roughly three in ten top-five slots are relevant ($\text{P@5} \approx 0.31$)—because the corpus is small, labels are sparse, and several jobs can appear plausible

Table 4: Paper progression benchmark (regression baselines, $K=5$).

Method	P@5	R@5	nDCG@5
Multimodal soft embed $w=0.7$	0.313	1.000	0.969
Soft embed + cross-encoder (pool=10)	0.307	0.983	0.939
RRF ensemble (4 lists)	0.293	0.950	0.935
Multimodal Jaccard $w=0.7$	0.293	0.950	0.933
Semantic cosine (rich templates)	0.300	0.950	0.922
Semantic cosine	0.280	0.900	0.911
TF-IDF (lexical)	0.307	0.983	0.905
BM25 (lexical)	0.307	0.983	0.901

Source: `data/expected/paper_progression.summary.json`. Produced by `benchmarks/paper_progression.py`; gated by `tests/benchmarks/test_eval_regression.py` (± 0.04 nDCG tolerance). Values can differ from Table 3 when template or cache flags differ.

Table 5: Fusion and constraint variants ($K=5$).

Method	P@5	R@5	nDCG@5
Learned fusion (LR)	0.307	0.983	0.968
Fixed multimodal ($\alpha=0.7$)	0.287	0.933	0.924
Hierarchical skills	0.287	0.933	0.924
Multimodal + constraints	0.300	0.950	0.908
Semantic cosine	0.267	0.867	0.878

Source: `backend/benchmark_outputs/table11.fusion.json` (`benchmarks/table11.fusion.py`).

Table 6: Fitted calibration and fusion model parameters (offline training).

Component	Value
<i>Platt calibrator</i> (<code>data/models/calibration.json</code>)	
a (scale)	2.078
b (shift)	2.112
<i>Learned fusion LR</i> (<code>data/models/fusion.json</code>)	
Top feature weights (logistic)	exp. 0.860, remote 0.853, salary 0.849 semantic 0.537, taxonomy 0.509, skills 0.204 must-have coverage 0.311, bias 1.529

Models trained on labeled eval pairs; used when `use_calibration` or `fusion_mode=learned` is enabled. *TODO*: Report calibration impact on nDCG@5 and expected calibration error.

for the same résumé. We therefore characterize JobMatch as *assistive* ranking with explainable reasons, not as an automated hiring decision system.

6.4 Multi-agent state sharing and usability

The offline tables above test the Matchmaking agent’s *scoring* logic. They do not directly score portal usability or quantify latency perceived by end users. We separate what the benchmarks show from what the architecture is designed to improve.

When a candidate edits a resume or an employer revises a job description, the owning agent rebuilds a versioned snapshot and shared embedding, then notifies the Matchmaking layer through the event bus (§3–§4). The event-driven path addresses a usability problem seen in monolithic ATS tools: rankings that stay stale after a profile change, or that change without a visible cause. Role-separated agents also keep each side’s data under its owner’s control, which supports trust and auditability even when two users see the same match score from the neutral Matchmaking agent.

State sharing therefore improves **system usability and traceability**—fresh rankings after edits, clear ownership, structured explanations—more than it maximizes a single offline metric on a frozen corpus. The benchmarks validate that the shared scoring core ranks sensibly under manual labels; they do not yet prove that multi-agent coordination reduces time-to-hire or increases applicant satisfaction in the field.

6.5 Strengths

Several patterns repeat across tables.

Hybrid semantic–skill ranking.. Any method that combines text meaning with skills beats semantic-only search (nDCG@5 0.878 vs. 0.917–0.942 for richer composites). Table 7 shows that dropping experience, compensation, or location *alone* as the only signal yields very poor rankings (nDCG@5 ≤ 0.39), but adding them alongside semantic and skill channels contributes to the full composite score (0.942 under the research weight set in that table).

Table 7: Ablation over composite matching components ($K=5$).

Variant	Category	P@5	R@5	MRR	nDCG@5	MAP
Full composite	Full	0.307	0.983	0.944	0.942	0.896
Semantic + skills	Partial	0.287	0.933	0.961	0.917	0.867
Semantic + skills + exp.	Partial	0.287	0.933	0.961	0.917	0.866
Semantic only	Single	0.267	0.867	0.931	0.878	0.810
Skills only	Single	0.233	0.733	0.816	0.748	0.681
Compensation only	Single	0.187	0.567	0.457	0.393	0.387
Location only	Single	0.167	0.467	0.388	0.335	0.328
Experience only	Single	0.160	0.467	0.384	0.326	0.314
Component RRF	Ensemble	0.233	0.717	0.570	0.564	0.497

Source: 2026-05-27T150000Z/ablation/ablation.summary.json. Weights for full composite in this run: semantic 40%, skills 30%, experience 15%, compensation 10%, location 5% (no title term). *TODO:* Re-run with portal weights 28/27/10/15/10/10.

Speed on a small pool.. Table 8 shows that lightweight scorers complete in well under one millisecond per resume on 15 jobs; even the full composite bi-encoder run

averages ~ 0.4 ms per query. That headroom matters for interactive portals where a user expects refreshed matches after saving a profile.

Table 8: Quality-latency trade-off (exhaustive evaluation, $K=5$).

Method	nDCG@5	Latency (ms/q)	Notes
Exact skill overlap	0.748	0.03	Lexical
Skills Jaccard	0.748	0.03	Set overlap
TF-IDF cosine	0.898	0.07	Lexical
BM25	0.894	0.11	Lexical
Semantic euclidean	0.878	0.21	Embedding
Semantic cosine	0.878	0.23	Embedding
Multimodal weighted	0.924	0.26	$\alpha=0.7$, Jaccard skills
RRF ensemble	0.913	0.95	Four ranked lists
Soft skill embed	0.869	15.53	Per-skill embedding
Composite (bi-encoder)	0.942	0.41	Ablation full composite
Composite + cross-encoder	0.834	141.7	Pool=20; Δ nDCG=-0.108

Latency from `comparison_summary.json` and `cross_encoder_report.json`. *TODO:* Publish `phase11.py` ANN shortlist latency table (pool=10, Chroma/Qdrant grid).

Label-set sanity.. Table 9 confirms that hard-negative mining—high-scoring jobs *not* in the labeled relevant set—never overlaps declared relevant jobs across 150 mined pairs. This does not prove those jobs are truly bad fits, but it shows the label file is internally consistent for benchmark purposes.

Table 9: Hard-negative mining summary (multimodal top-10 pool, $K=5$ negatives per query).

Statistic	Value
Queries mined	30
Negatives per query (target / observed)	5 / 5
Total hard-negative pairs	150
Negatives with graded relevance > 0	0
Negatives overlapping declared relevant set	0

Source: `backend/benchmark_outputs/hard_negatives.json`. Mined with multimodal weighted scoring ($\alpha=0.7$) over 15 jobs; negatives are high-scoring non-labeled jobs, not confirmed irrelevant documents.

Explainability path.. The live portal defaults to composite scoring precisely because each channel maps to a human-readable explanation (skills matched, experience gap, remote fit). Research drivers achieve slightly higher nDCG with soft-embed multimodal or learned fusion, but those variants expose fewer interpretable components to end users (§4.5).

6.6 Limitations

The following limits follow from the corpus size, method choices, and scope of measurement.

Small, controlled dataset.. Thirty resumes and fifteen jobs make recall@5 easy to saturate—many methods reach ≥ 0.93 —so differences in nDCG are informative but narrow. Results may not transfer to employers with thousands of open roles or to domains unlike our demo profiles.

Precision still modest.. Even the best methods place relevant jobs in only about three of every ten top-five slots ($P@5 \approx 0.31$). Users will still see imperfect suggestions and must rely on explanations and their own judgment.

Configuration mismatch.. Offline ablations use research composite weights (40/30/15/10/5) while the portal uses 28/27/10/15/10/10 with a title channel. We have not yet published ablation numbers under portal-default weights *TODO*: Publish composite ablation under portal weights for deployment alignment..

Advanced scorers that disappoint offline.. Cross-encoder reranking is $\sim 340\times$ slower than composite bi-encoder scoring on this setup and *lowers* nDCG@5 from 0.942 to 0.834 in the committed report—consistent with leaving it disabled by default. Learned fusion and calibration fit on the same labeled pairs they are judged against, which risks overfitting.

Fairness scope is narrow.. Table 10 runs a synthetic counterfactual audit on ten fabricated profile pairs (name, summary, or metadata edits). Six pairs trigger flags—mostly small rank shifts or score changes when non-qualification text changes—with a maximum score shift of 0.024. This is an engineering sanity check, not demographic fairness certification; proxy-group disparate-impact ratios are not yet committed to benchmark artifacts *TODO*: Export proxy-group disparate-impact ratios from `fairness_eval.py`..

Table 10: Synthetic counterfactual fairness audit (10 pairs, composite ranking).

Pair	Field changed	Top-1 stable	Top-5 overlap	Max rank Δ	Max score Δ	Flagged
name_gender_01	name	Yes	5/5	1	0.0096	No
name_gender_02	name	Yes	5/5	0	0.0072	No
name_ethnicity_01	name	Yes	5/5	2	0.0157	Yes
name_ethnicity_02	name	Yes	5/5	3	0.0238	Yes
nationality_phrase_01	summary	No	5/5	1	0.0087	Yes
nationality_phrase_02	summary	Yes	5/5	1	0.0110	No
hometown_01	hometown	Yes	4/5	2	0.0117	Yes
hometown_02	hometown	Yes	5/5	2	0.0143	Yes
pronouns_01	pronouns	Yes	5/5	1	0.0042	No
email_domain_01	email	Yes	4/5	1	0.0082	Yes

Source: `docs/research/evaluation/paper_tables/table4_fairness.csv`. Fabricated profiles only; 6/10 pairs flagged. *TODO*: Export proxy-group disparate-impact ratios from `fairness_eval.py`.

No real hiring outcomes.. We did not measure interview rates, time-to-fill, applicant satisfaction, or recruiter workload. Explainability quality is defined offline (§5.5) but not yet summarized in a results table here *TODO*: Add explainability evaluation summary table to this section..

6.7 Implications for users

Job seekers.. JobMatch is designed as an assistive representative: maintain a structured profile, inspect why a job ranked highly or poorly, and decide whether to save or apply. The evaluation indicates that suggestions are most reliable when the system weights both résumé text and listed skills—consistent with how applicants skim postings for fit. Top-five lists are neither exhaustive nor authoritative; with P@5 near 0.31, several shown jobs may be weak fits, and relevant openings can be missed when the job pool exceeds fifteen roles.

Employers.. Employers receive symmetric reverse matching: ranked candidates with component-level reasons instead of an opaque score. Hybrid ranking supports recruiters who weight keywords and hiring managers who read career narrative. Hard post-filters can remove partially suitable candidates from the graded top five, so the portal keeps constraints optional and off by default. When a job description changes, the Employer Agent update path refreshes rankings without re-upload to a separate scoring service.

Shared implication.. Both sides benefit when software explains its suggestions and leaves hire, reject, and apply decisions with users. The benchmarks support that design on a demo corpus: combined signals rank better than single-signal search, explanations align with composite channels, and the multi-agent layout keeps profile edits on the owning side while a neutral Matchmaking Agent serves both portals. They do not establish production gains on hiring outcomes; that requires larger labeled datasets, portal-default ablations, and field studies.

7 Conclusion and Future Scope

7.1 Conclusion

Job seekers and recruiters still interact through software that ranks quickly but explains little. When a résumé or job description changes, the other party typically sees a new list without a traceable link to the edit. Candidates may prepare for interviews on weak matches; recruiters may shortlist profiles without seeing which requirements drove the order. The underlying problem is therefore one of *trust and accountability*: both sides need clear record ownership, evidence for each match, and explicit control over consequential actions.

JobMatch addresses that problem through a multi-agent architecture rather than a monolithic applicant-tracking matcher. Three agents divide the work by role. The **Candidate agent** accepts résumés, builds a structured profile and embedding, and refreshes that state when the job seeker edits their materials. The **Employer agent** does the same for job descriptions: parse postings, version requirements, and update vectors when a role changes. The **Matchmaking agent** remains read-only with

respect to both stores: it reads versioned snapshots from each side, scores candidate–job pairs, and returns ranked lists with structured explanations. Candidate, employer, and admin portals sit on top of this layout so that each party manages its own data while sharing only what matching requires.

We evaluated the Matchmaking agent offline on a labeled demo corpus—30 résumés, 15 jobs, and 47 graded relevance pairs—using the metrics defined in §5 and reported in §6. Hybrid rankers that combine semantic text similarity with explicit skill overlap beat semantic search alone; on this small pool, keyword methods such as BM25 remain competitive. The best research configurations reach $\text{nDCG@5} \approx 0.97$ and $\text{recall@5} = 1.00$, but precision@5 stays near 0.31, so even strong setups surface imperfect top-five lists. The live portal therefore defaults to a fixed composite score whose channels map directly to user-visible explanations, rather than to the single highest offline nDCG variant. These results are reproducible from committed benchmark artifacts; they characterize ranking behavior on a controlled corpus, not hiring quality in production.

JobMatch is intentionally built as **decision-support software**, not an automated hiring replacement. It suggests, ranks, and explains; users save, apply, shortlist, or pass—the system does not hire, reject, or negotiate on anyone’s behalf. The contribution is an integrated research prototype: role-aware agents, shared snapshots, explainable composite ranking, role-separated portals, optional external job-ingestion for deployment, and offline evaluation tied to the same scoring code the gateways invoke. Whether this design improves real-world trust, time-to-shortlist, or fairness is not settled by laboratory metrics alone and remains for field study.

7.2 Future scope

The following directions extend JobMatch while preserving human authority over consequential hiring actions.

Stronger LLM-based résumé and job-description parsing.. Today’s prototype uses lightweight JSON-oriented parsing (§4). Large language models could extract skills, seniority, and requirements from noisy PDFs and informal postings, with users confirming fields before snapshots enter the vector store.

Cross-encoder reranking.. A cross-encoder second stage exists in the codebase but is off by default: on the demo corpus it added roughly two orders of magnitude latency while *reducing* nDCG@5 (§6). Future work should retest reranking on larger corpora and tighter shortlists, where gains at rank 1 may justify the extra cost.

Larger real-world datasets.. A synthetic scale corpus already lives under `data/research/` (100 candidates, 50 jobs, 5000 labeled pairs). The prototype also ships an optional external job-ingestion path (`/real-jobs/*`) that can refresh the Employer agent corpus from a provider API while offline benchmarks continue to use the frozen `jobs.json` demo set (§4). The next step is held-out evaluation on real or expert-labeled résumé–job pairs, including composite ablation under portal-default weights (28/27/10/15/10/10) *TODO*: Run composite ablation with portal weights 28/27/10/15/10/10..

Production hardening for live job feeds.. External sync is implemented as administrator-triggered HTTP endpoints and a CLI script; future work should add

authenticated admin UI controls, rate limiting, and provider credential management before exposing sync in untrusted deployments.

Human evaluator studies.. Offline precision and nDCG do not measure whether explanations help people decide faster or with more confidence. Structured studies with recruiters and job seekers—testing comprehension, trust, and behavior after viewing match reasons—are needed before deployment claims.

Fairness and bias auditing.. The current synthetic counterfactual audit detects rank instability under controlled profile edits but is not demographic fairness certification (§6). Future work should expand audit suites, commit proxy-group disparate-impact ratios, and publish explanation-quality summaries alongside ranking tables *TODO*: Commit `fairness_eval` JSON exports and explanation-quality tables..

Privacy-preserving profile sharing.. Separate agent ownership raises consent and minimization questions: which fields cross the Matchmaking boundary, when embeddings suffice instead of raw text, and how updates propagate with auditable permission. Field-level redaction, encrypted snapshot exchange, and privacy-preserving aggregate feedback deserve systematic study.

Recruiter and candidate feedback loops.. The platform logs match feedback (save, dismiss, apply) but disables score boosting from that history in default portal settings (§4.5). Future work can evaluate boost policies offline, keep humans authoritative over outcomes, and measure whether logged preferences improve relevance without obscuring reasons.

Deployment-scale latency evaluation.. Interactive matching must stay responsive as job pools grow from dozens to thousands of roles. ANN shortlist sweeps (`phase11.py`) should produce committed latency tables that separate vector retrieval from pairwise scoring on Chroma and Qdrant backends *TODO*: Commit `phase11_summary.csv`..

The design goal is unchanged: agents keep profiles current, the Matchmaking layer shows its work, and people retain final hiring decisions.

Acknowledgments. The authors thank Dr Parteek Bhatia (Washington State University) for supervision and guidance, and Thapar Institute of Engineering and Technology for institutional support. Collaborators who reviewed portal workflows and benchmark reproducibility checks are also acknowledged.

Declarations

Funding

Not applicable.

Competing interests

The authors declare no competing interests.

Ethics approval

Not applicable. Evaluation uses synthetic and manually curated demo profiles; no human subjects recruitment data were collected for this study.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Data availability

The demo corpus (`data/cvs.json`, `data/jobs.json`, `data/eval_pairs.json`) and benchmark outputs under `backend/benchmark_outputs/` and `docs/research/evaluation/` are included in the project repository. Regenerate metrics with `python backend/scripts/run_research_pipeline.py` and `python -m benchmarks.paper_progression`.

Code availability

Source code for the platform, benchmark drivers, and regression tests is available in the JobMatch repository at the submission tag. Key entry points: `backend/gateway/`, `backend/agents/`, `backend/core/real_jobs_sync.py`, `frontend/src/`, `bash scripts/run_tests.sh`, and `tests/benchmarks/test_eval_regression.py`.

Author contributions

Harsh Kashyap and **Taranumpreet Kaur Wasu** contributed equally to conceptualization, software implementation, evaluation, and manuscript preparation.

Prior publication

This manuscript has not been published previously and is not under consideration elsewhere. No portion of the text has appeared in a preprint with colored layout styling.

References

- [1] Khan, S.: Brave New Words: How AI Will Revolutionize Education (and Why That’s a Good Thing). Viking, ??? (2024)
- [2] Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press, ??? (2008)

- [3] Robertson, S., Zaragoza, H.: The probabilistic relevance framework: BM25 and beyond. In: Foundations and Trends in Information Retrieval, vol. 3, pp. 333–389 (2009)
- [4] Wooldridge, M.: An Introduction to MultiAgent Systems, 2nd edn. Wiley, ??? (2009)
- [5] Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: Proceedings of EMNLP-IJCNLP, pp. 3982–3992 (2019)
- [6] Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal rank fusion outperforms condorcet and individual rank learning methods. Proceedings of SIGIR, 758–759 (2009)
- [7] Platt, J.: Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Advances in Large Margin Classifiers, 61–74 (1999)
- [8] Ribeiro, M.T., Singh, S., Guestrin, C.: “why should i trust you?”: Explaining the predictions of any classifier. Proceedings of KDD, 1135–1144 (2016)